Composer & piloter la boucle

J2 matin

Workflows, supervision et loops, sur votre projet ou sur Tailspin Toys

Émargement de vendredi matin : signez en arrivant

`edusign.app/sc/BP0FZ4UR`

EXERCICE Votre projet, ou Tailspin Toys

Un projet sur lequel avancer aujourd'hui ? Restez dessus. **Sinon**, installez Tailspin Toys :

```
git clone https://github.com/denneulin/tailspin-toys.git
cd tailspin-toys
npm ci
npm run dev        # http://localhost:4321
```

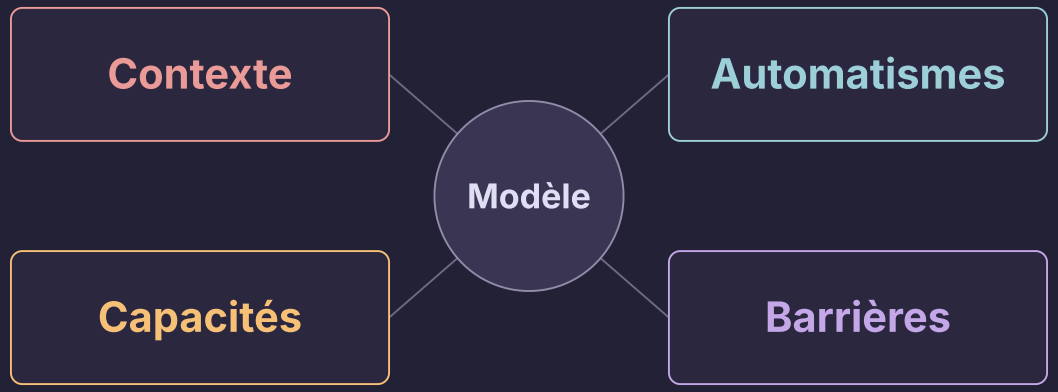
Puis, pour tous, dans Claude Code lancé dans votre projet :

```
/plugin marketplace add mattpocock/skills
/plugin install mattpocock-skills@mattpocock
```

Pendant que ça s'installe, on continue.

Git, Node 24, sans droits d'administrateur, partage d'écran : **la page « Démarrer le J2 »**

Hier, aujourd'hui



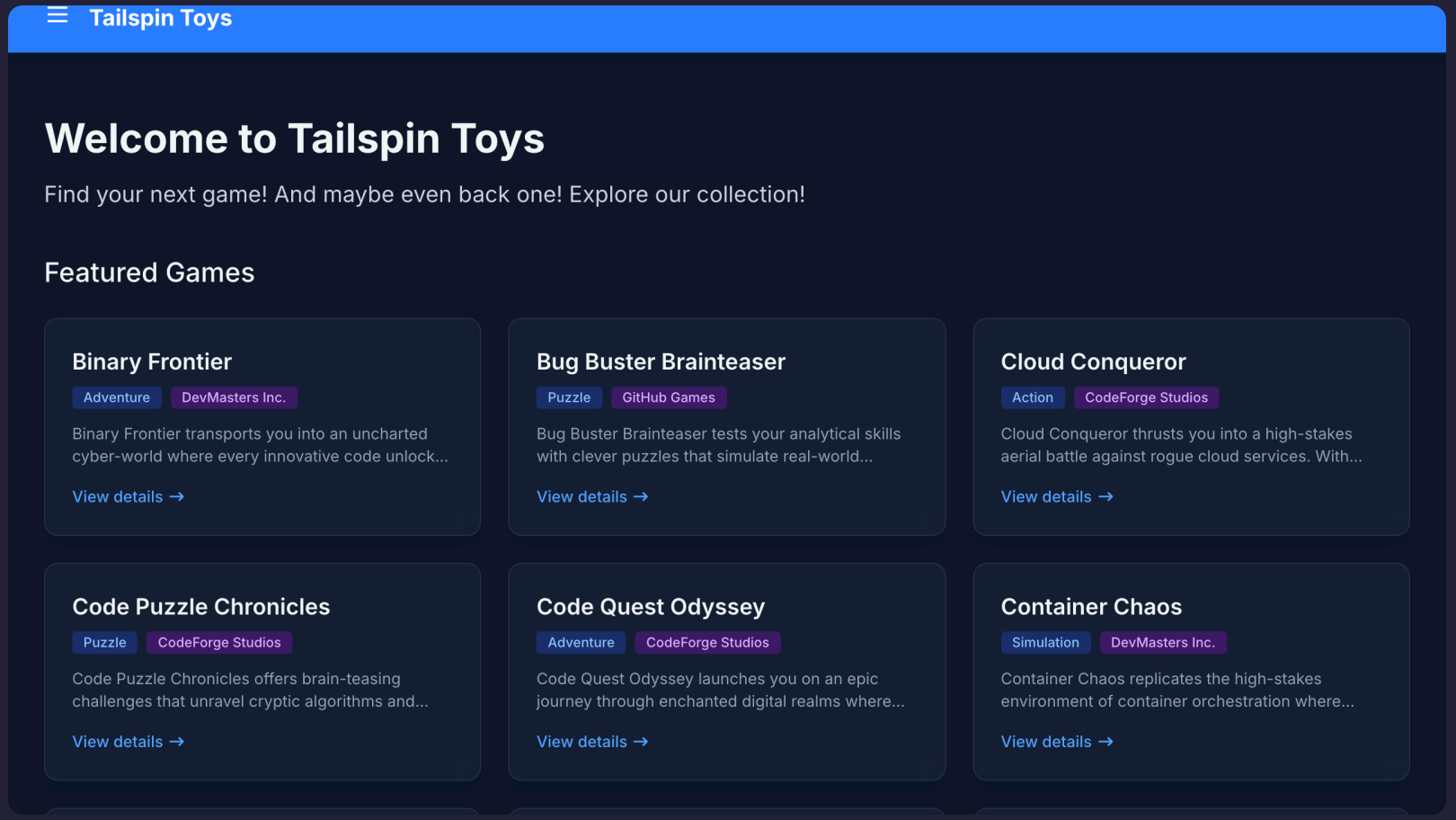
Hier : le harness configuré

Contexte, CLAUDE.md et règles, hooks, skills

Aujourd'hui

- Composer un **workflow**
- Choisir son niveau de **supervision**
- Lancer une **loop** qui s'arrête toute seule
- Cet après-midi : **paralléliser**

Tailspin Toys



Un site fictif de financement participatif de jeux, tiré des ateliers d'agents de GitHub, préparé pour Claude Code. Le reste, demandez-le à Claude.

EXERCICE Faites-vous présenter le projet

Lancez Claude Code dans `tailspin-toys`, et demandez-lui de vous présenter le projet, **à votre façon** : ce que vous demanderiez à un collègue, le premier jour.

Ne modifiez rien.

Sur votre projet : lancez `/setup-matt-pocock-skills` sur une branche, et choisissez les fichiers Markdown locaux.

4 minutes

EXERCICE

Puis ce prompt

```
Ne modifie rien. Présente-moi ce projet en cinq lignes : ce qu'il fait,
sa stack, comment on lance les tests. Puis résume chaque ticket de
backlog/ en une ligne : ce qui change pour l'utilisateur.
```

Comparez avec ce que vous avez obtenu à votre façon : qu'est-ce que le prompt a changé ?

3 minutes

EXERCICE Votre ticket du jour

Votre ticket	Où le trouver
Un ticket de votre projet, perso ou client	votre backlog
La 07, proposée à tous sur Tailspin : filtrer les jeux par catégorie et par éditeur	`backlog/07-filter-games.md`
Un autre ticket du backlog de Tailspin, de 01 à 06	`backlog/` et le README
Votre idée sur Tailspin : une liste de souhaits, des avis…	une phrase à vous

Écrivez-le **en une phrase** dans le cahier, puis tour rapide : **15 secondes chacun**.

Ce matin, sa spec et ses tickets · avant le déjeuner, une loop sur le premier · cet après-midi, les suivants en parallèle · en fin de journée, ce que vous en tirez

Workflows

Composer plutôt qu'adopter

Adopter une méthode, ou composer ses skills

Adopter

Une méthode clé en main (GSD, BMAD, Spec-Kit…) **possède le processus** : ses étapes, ses gabarits, ses agents.

Rapide à démarrer ; **difficile à corriger** quand une étape ne vous va pas.

Composer

De **petits skills**, une responsabilité chacun, enchaînés par vous ou par un script.

Vous gardez **la main sur chaque étape** : une étape vous gêne, vous changez un skill.

Une méthode s'adopte en bloc ; un workflow se compose.

Source : Matt Pocock, README de github.com/mattpocock/skills

La spec, source de vérité

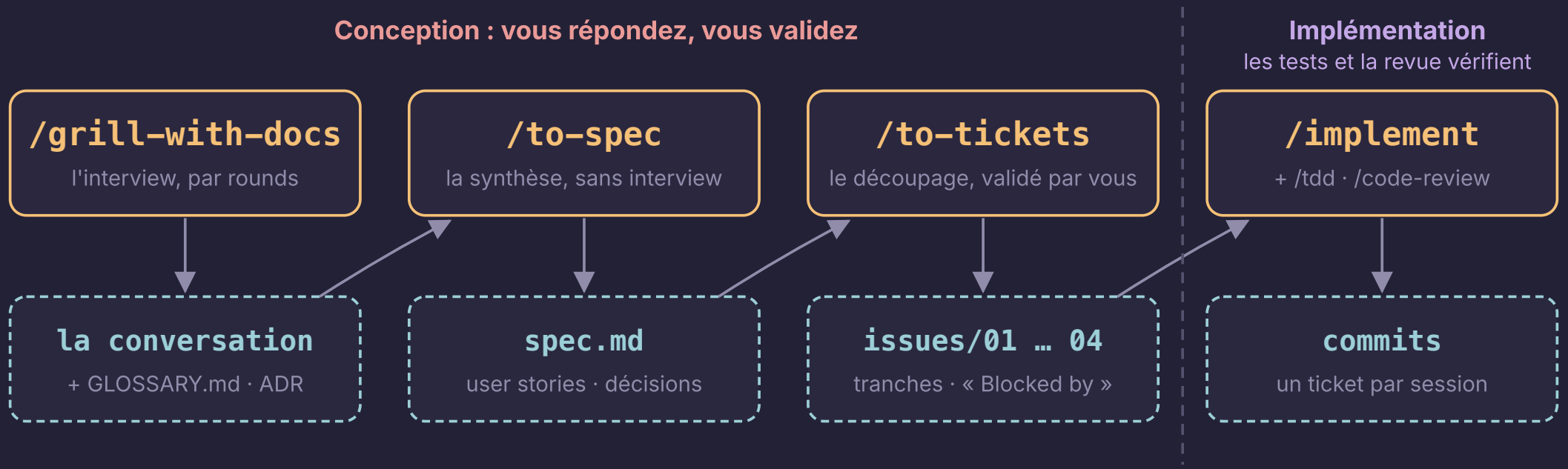
Niveau	La spec…	Exemple
Spec-first	écrite avant, sert à la tâche en cours	Spec-Kit, Kiro
Spec-anchored	gardée après la tâche, sert à faire évoluer la fonctionnalité	une spec versionnée à côté du code
Spec-as-source	la seule chose qu'on édite ; le code en découle	Tessl (bêta privée en 2025)

La doc Claude Code : se faire **interviewer**, écrire la spec dans un fichier, **puis ouvrir une nouvelle session** pour l'exécuter.

Une spec écrite survit au `/clear` ; une conversation, non.

Sources : Birgitta Böckeler, martinfowler.com, 15/10/2025 ; doc Claude Code, Best practices

Une chaîne de skills : interview → spec → tickets



Interview, spec, tickets : une même conversation. Puis chaque ticket, en contexte frais.

Source : github.com/mattpocock/skills (licence MIT)

`/grill-with-docs` : une interview par rounds

tailspin-toys — claude

Read **1** file, listed **2** directories, ran **3** shell commands

● Ticket, schéma, couche de données, page d'accueil et règles du projet lus. Je démarre l'entretien.

Faits établis (pas besoin de vous les demander)
- Le catalogue compte **21 jeux**, tous notés. La note est `star_rating`, un `real` qui peut être nul. Le seed la tire de `ratingFromTitle`, dans [3.0, 5.0]. La moyenne réelle est de **3,905**.
- Il n'y a ni `GLOSSARY.md` ni `docs/adr/`. Je les créerai au fil de l'entretien, quand un terme sera tranché.
- `src/pages/index.astro` charge déjà `getAllGames()` et affiche `EmptyState` quand il n'y a aucun jeu.
- Un jeu sans note s'affiche déjà « Not yet rated » (`formatStarRating`), et `StarRating` borne la note avec `clampRating` avant de l'afficher avec `toFixed(1)`.
- D'après `.claude/rules/donnees.md`, une fonction d'accès aux données reçoit `db` en premier paramètre et se teste sur une base en mémoire, sans mock.

? **Q1 — Le mot « catalogue » :** le ticket dit *catalog*, le sous-titre de l'accueil dit *collection*, et le titre de section *Featured Games*. Le catalogue, est-ce **tous les jeux de la base**, sans exception (aujourd'hui, rien ne permet d'exclure un jeu : pas de statut, pas de jeu « publié ») ? Et faut-il écarter *collection* comme synonyme ?

→ Oui. **Catalogue** = l'ensemble des jeux proposés sur le site, c'est-à-dire tous les jeux de la base. On écarte *collection* et *inventaire*. « Featured Games » reste un libellé d'interface et n'entre pas dans le glossaire.

Un vrai round, sur la 05 de Tailspin : les faits, il les trouve ; les mots, vous les tranchez.

ATELIER 2 De votre ticket à une spec et des tickets

Le livrable

La spec de votre ticket, vos tickets avec leurs dépendances, et votre plan de composition.

Le déroulé

58 minutes, seul, chacun son ticket. On avance **ensemble, étape par étape** : chaque étape s'affiche quand on y arrive.

Votre ticket sous les yeux : on commence.

ATELIER 2 · ÉTAPE 1/4

Se faire interviewer

```
git switch -c <votre-ticket>     # 07-filtres pour la 07
/model opus
/grill-with-docs <votre ticket : le chemin du fichier, ou votre phrase>
```

- **Tranchez les termes** qu'il vous propose : ils partent dans `GLOSSARY.md`.
- **Répondez vraiment** quand une recommandation ne vous va pas : le product owner, c'est vous.
- Quand il résume et vous demande s'il peut y aller : **ne dites pas oui**.

Fini en avance ? Demandez-lui : « Quelles décisions as-tu prises à ma place ? », puis relisez `GLOSSARY.md`.

23 minutes

ATELIER 2 · ÉTAPE 2/4

La spec

```
/to-spec Dis-moi d'abord où tu comptes tester, et attends mon accord avant d'écrire.
```

- Il **n'interviewe plus** : il synthétise votre conversation, explore le code et vous propose ses **points de test** (seams), là où ses tests attaqueront le code : une fonction, une route, une page. Validez-les ou corrigez-les.
- Dans `spec.md`, lisez surtout **Implementation Decisions** et **Out of Scope**.

Fini en avance ? Relisez Out of Scope : ce qui y manque, faites-le ajouter.

9 minutes

ATELIER 2 · ÉTAPE 3/4

Les tickets

```
/to-tickets
```

- Demandez **un premier ticket qui en débloque deux** : une loop fera le premier avant le déjeuner, les deux suivants partiront en parallèle l'après-midi. Sur la 07, le ticket le prévoit déjà : vérifiez-le.
- **Refusez un découpage par couche** : une tranche verticale par ticket, vérifiable seule.

Fini en avance ? Relisez vos tickets : une tranche verticale ? vérifiable seule ? les « Blocked by » justes ?

11 minutes

ATELIER 2 · ÉTAPE 4/4

Le plan de composition

Ticket	Bloqué par	Skills qui existent	Skills à créer	Vérification
01 Filtrer par une catégorie, de bout en bout	—	`/tdd`, `/implement`	un skill « filtre de liste », si ça revient	`npm run test:unit`
02	.....	.....	.....	.....

Dans le cahier, une ligne par ticket · 9 minutes

C'est le livrable 3 du programme, et la carte de l'Atelier 3.

Restitution express

1

Votre ticket **en une phrase**

2

Combien de tickets, et lesquels peuvent partir **en parallèle** ?

3

Une question de l'interview qui vous a **surpris**

3 volontaires, 90 secondes chacun.

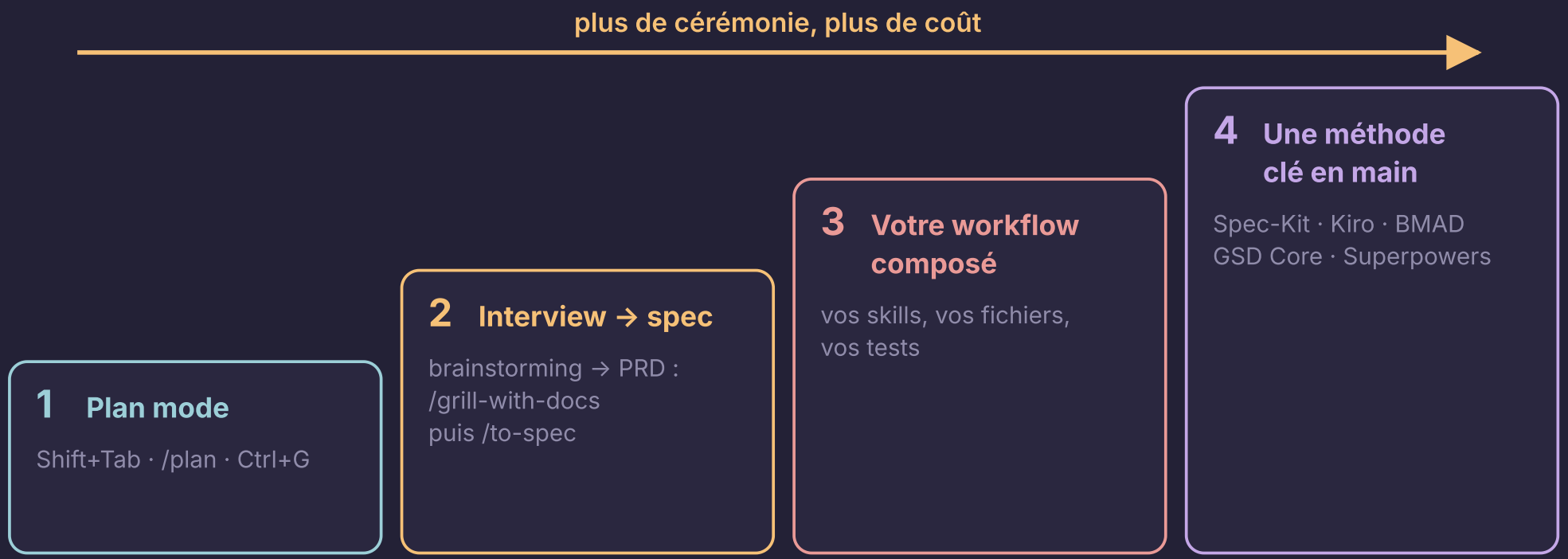
Pause

Reprise dans 15 minutes

Doser

Plan mode, PRD, méthodes

Du plus simple au plus outillé



Si le diff tient en une phrase, pas de plan.

Les méthodes clé en main, et leurs limites

Méthode	Le principe	Ce qu'on lui reproche
Spec-Kit (GitHub)	une constitution, puis specify → plan → tasks → implement → converge	une même cérémonie pour toutes les tailles ; le retour du « waterfall »
Kiro (AWS)	un agent qui produit requirements, design et tasks	idem
BMAD	des agents analyste, PM, architecte, dev, UX	lourd, courbe d'apprentissage raide
GSD Core	discuss → plan → execute → verify → ship, en sous-agents	peine sur des besoins ambigus ou mouvants
Superpowers	des skills composables : brainstorming, plans, TDD, review	coût en tokens ; redondant avec le plan mode et les sous-agents

Et à toutes : des plans très longs, que personne ne relit. On y pioche des idées ; on ne les adopte pas en bloc.

Sources : dépôts et sites des projets, état au 14/10/2026 ; Böckeler (martinfowler.com, 10/2025) ; Marmelab (11/2025) ; Hicron (05/2026) ; MCP.Directory (06/2026)

Ce qu'on garde

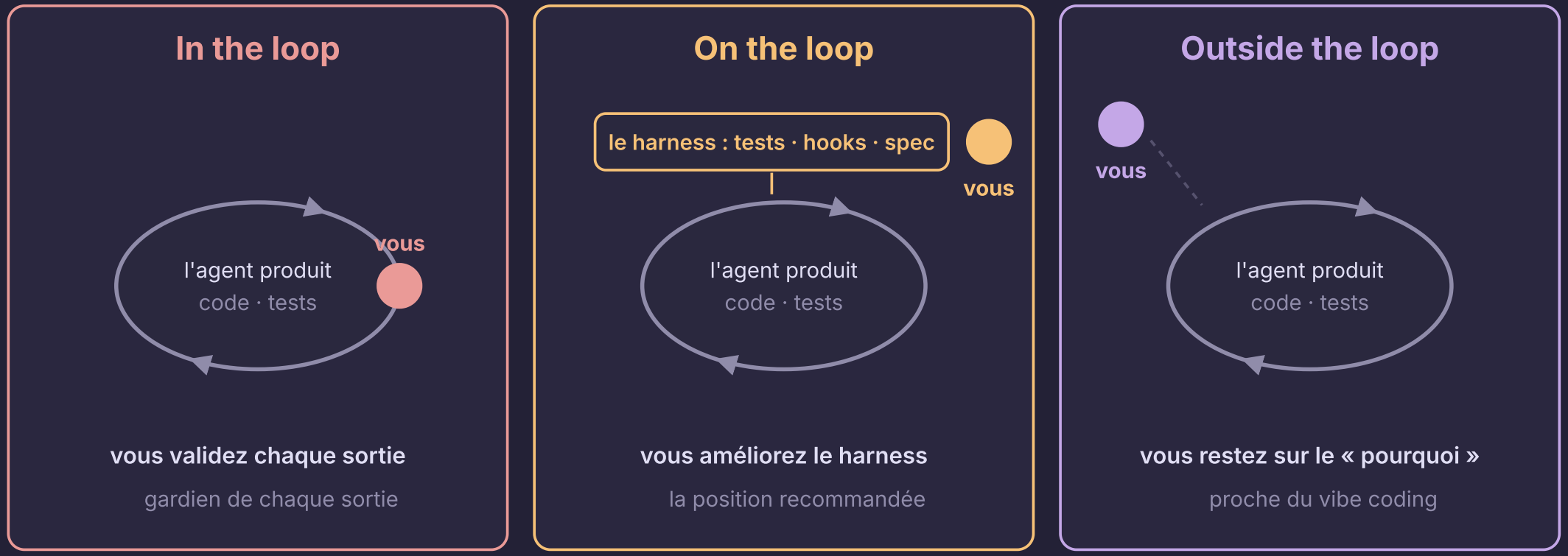
Principe	La brique
Des **skills atomiques**	une responsabilité chacun
Un **fichier court** entre deux étapes	la spec, les tickets, le plan
Un **contexte frais** par phase	`/clear`, une nouvelle session, un sous-agent
Des **vérifications automatiques**	les tests, les hooks, `/goal`
Un **séquenceur**	vous, un skill qui orchestre, un script

Cet assemblage, c'est votre harness.

Supervision

In, on, outside the loop

Trois positions



Le spectre de la délégation : du développeur qui relit chaque sortie à l'orchestrateur qui améliore le harness.

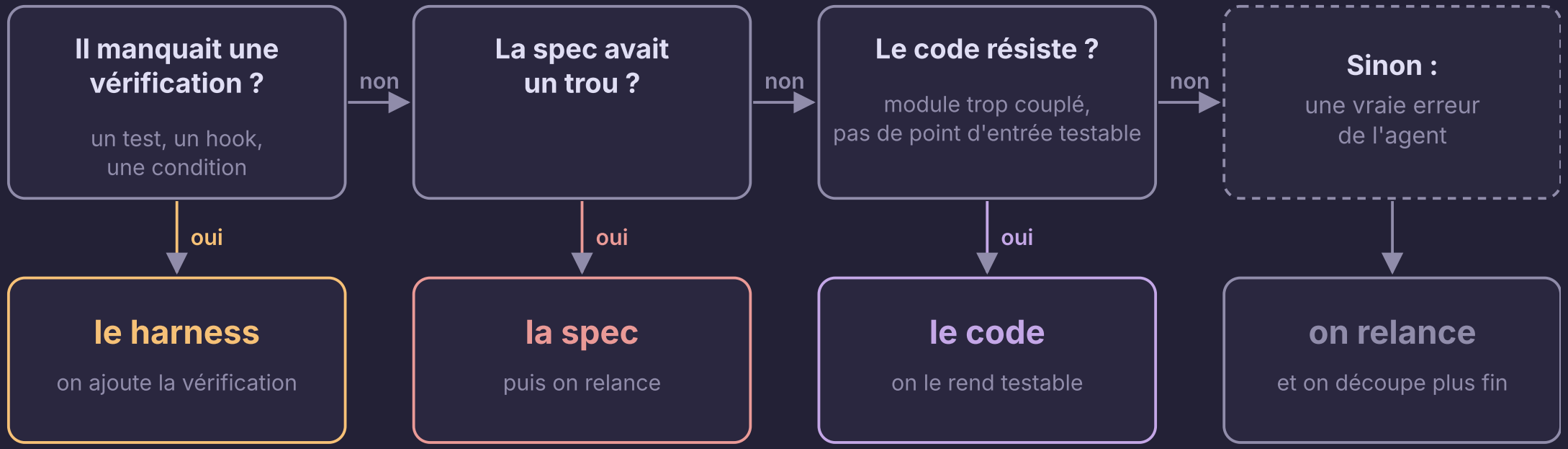
Source : Kief Morris, « Humans and Agents in Software Engineering Loops », martinfowler.com, 04/03/2026

Le niveau dépend de la tâche

Étape ou tâche	Niveau	Pourquoi
Conception : spec, découpage, critères	**in the loop**	les décisions sont les vôtres
Implémentation d'un ticket bien spécifié et testé	**on the loop**	les tests, les hooks et `/goal` vérifient pour vous
Décider du merge	**in the loop**	sur preuves : tests verts, rapport, relecture en contexte frais
Prototype jetable	outside, à la rigueur	rien à maintenir
Migration de base de production, sécurité	**in the loop**	l'erreur coûte trop cher

Monter d'un cran demande un harness plus solide, pas plus de courage.

Quand vous rentrez dans la boucle



Corriger la sortie, c'est rester dans la boucle pour toujours.

Rendre la vérification obligatoire



Plus vous lâchez la main, plus la vérification doit être automatique.

D'après la documentation Claude Code, Best practices : « Each step trades setup for attention. »

Les loops

Ce qui les lance, ce qui les arrête

Les 4 types de loop

Turn-based

lancée par
votre prompt



s'arrête quand
Claude juge que c'est fini

une conversation normale

Goal-based

lancée par
une condition

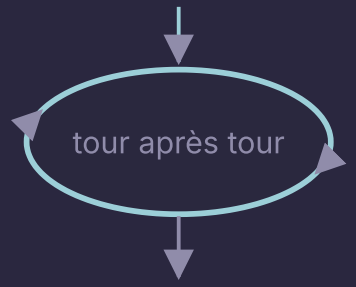


s'arrête quand
la condition est atteinte
(ou le plafond dépassé)

`/goal`

Time-based

lancée par
un intervalle



s'arrête quand
vous l'arrêtez,
ou le travail aboutit

`/loop`, `/schedule`

Proactive

lancée par
un événement ou un planning

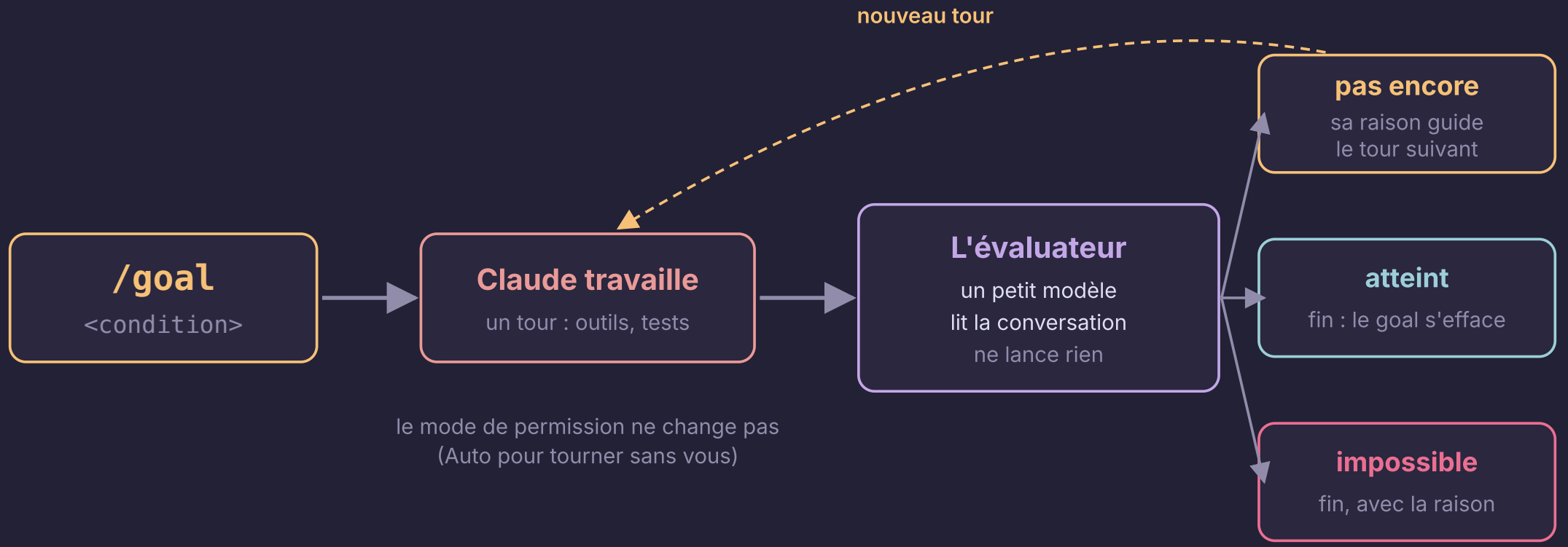


s'arrête quand
chaque tâche : son but atteint
la routine : vous la coupez

Routines, tâches planifiées du desktop, GitHub Actions

Sources : Anthropic, « Loop engineering: Getting started with loops », claude.com/blog, 30/06/2026 ; doc « Scheduled tasks »

`/goal` : une condition, un évaluateur



Ce n'est plus le modèle qui travaille qui décide qu'il a fini.

EXERCICE `/goal` à blanc

```
/clear
/goal npm run test:unit passe ; sinon arrête-toi après 2 tours
```

Regardez **combien de tours** il lui faut, et **ce qu'il a écrit**. Puis **Ctrl+O** : la raison du verdict.

Sur votre projet : votre commande de tests, à la place de `npm run test:unit`.

4 minutes · qu'est-ce qui cloche ?

Écrire une condition d'arrêt

Floue : « /goal répare le module commandes »

Vérifiable :

```
/goal `dotnet test --filter Category=Orders`
  passe sans échec ;
  aucun fichier de test existant n'est modifié ;
  sinon arrête-toi après 15 tours.
```

Une vérification énoncée
la commande, dont la sortie s'affiche

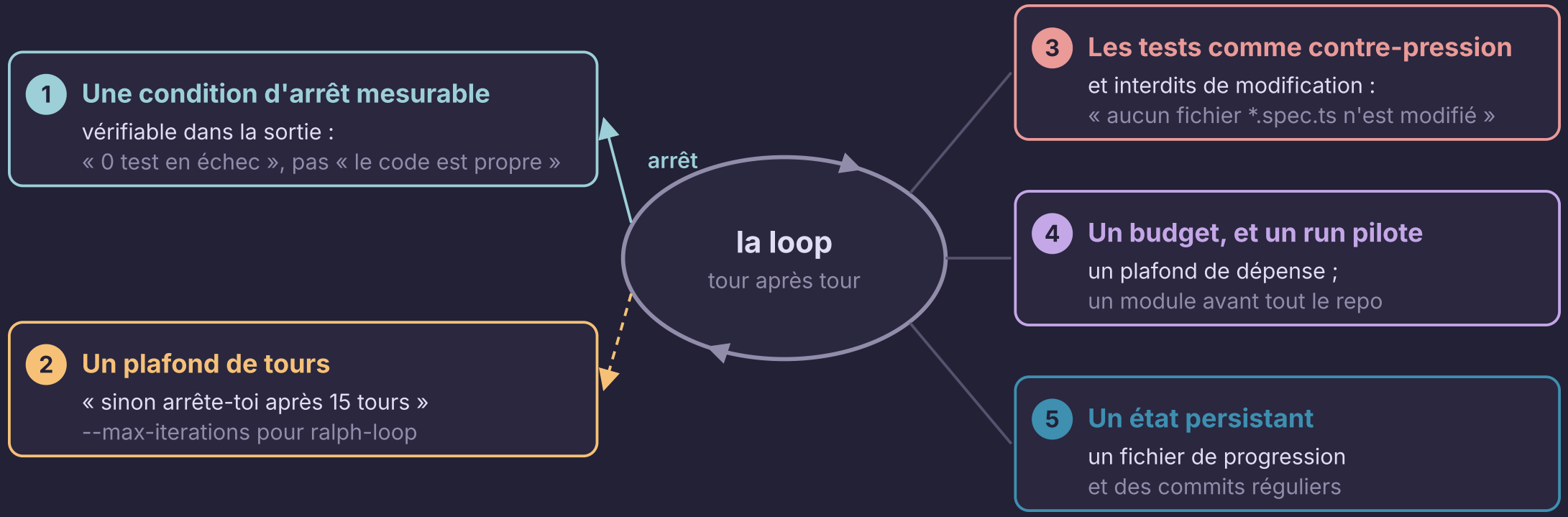
Un état final mesurable
« passe sans échec »

Les contraintes qui comptent
les tests ne bougent pas

Et un plafond
15 tours au plus

Les trois ingrédients d'une condition, d'après la documentation de `/goal`, et le plafond qu'elle recommande

Les 5 garde-fous d'une loop fiable



Cités sans être comptés : le sandbox, et un relecteur à contexte frais.

Sources : Anthropic (billets sur les loops et les long-running agents) ; Simon Willison ; Geoffrey Huntley ; doc /goal

EXERCICE La condition d'arrêt de votre ticket

1. Prenez le **premier ticket** de votre graphe : celui qui n'est bloqué par rien.
2. Écrivez sa **condition** `/goal` dans le cahier : un état final mesurable, une vérification énoncée, les contraintes, **et un plafond**.
3. **Ne la lancez pas encore** : on la lance ensemble juste après.

```
/goal <le nouveau test du ticket, nommé> passe,
  et npm run test:unit, npm run lint et npm run typecheck passent ;
  aucun autre fichier de test n'est modifié, ce que montre git status ;
  sinon arrête-toi après <n> tours.
```

Relisez-la : l'évaluateur pourrait-il trancher en lisant seulement la sortie de Claude ? À la répétition, il a validé une condition en notant « ...but the condition does not require that rendering check ». Il juge ce que vous avez écrit.

Fini en avance ? Écrivez celle du ticket suivant de votre graphe : elle servira cet après-midi.

6 minutes

EXERCICE Lancez-la

1. Sur votre branche, vérifiez le **mode Auto** : ▸▸ auto mode on dans la barre d'état ; sinon, Shift+Tab.
2. Collez votre condition : `/goal …`
3. `/goal` affiche le statut ; **Ctrl+O**, la raison du verdict.

Elle tourne pendant le déjeuner : session ouverte, ordinateur branché, sans mise en veille, et pas de `/clear`.

5 minutes · le verdict à la reprise

Et les autres loops

/loop

Time-based

```
/loop 5m vérifie si le
  déploiement est terminé
```

Tant que la session reste ouverte, 7 jours au plus.

ralph-loop

Plugin officiel

Le même prompt, relancé jusqu'à une phrase de fin :

```
/ralph-loop "<prompt>"
  --max-iterations 20
  --completion-promise "FINI"
```

Sans `--max-iterations` : illimité.

Routines

Proactive

Les Routines (cloud) et les tâches planifiées du desktop tournent sans vous, sur un planning ou un événement.

Vos questions

Le parking du matin

Plan d'adoption 30 jours : ligne 3

Ce que je mets en place dans mon équipe pour composer et piloter

J1 matin ligne 1 : CLAUDE.md et une règle
J1 après-midi ligne 2 : un hook, un skill
J2 matin **ligne 3 : composer et piloter**

Exemples : « une chaîne interview → spec → tickets sur notre prochain ticket » ; « une condition `/goal` écrite pour chaque tâche longue ».

Cet après-midi

Worktrees

1 worktree =
1 agent
= 1 tâche

Sous-agents

Un contexte frais, un résumé en retour

Atelier 3

Vos tickets, en parallèle

MCP

Brancher Claude sur un outil, sur Tailspin

Gardez vos tickets ouverts : on les lance cet après-midi.